SAIL-PROXY User Guide - Chapter 12

st-gr

2026-09-07

SAIL-PROXY User Guide

Multi-provider AI Gateway for SAP AI Core **Author:** *st-gr*

<< Previous Chapter | Content Table | Next Chapter >>

Using with Gemini CLI and the Gemini SDK

SAIL-PROXY speaks the Gemini API under `/google`, so any client built for Gemini — Google's **Gemini CLI**, the **`@google/genai`** SDK, or anything else that takes a Gemini base URL — can point at the gateway and reach **every model in the Model Library**, not only Gemini models.

Prerequisites

- **SAIL-PROXY installed and running** (see Installation).
- A **gateway API key** (create one via the CLI or Admin Cockpit — see Admin Cockpit).
- **Gemini CLI** installed (`npm install -g @google/gemini-cli`) and/or the **`@google/genai`** package for your own code.

Quick start: launch with the SAIL-PROXY CLI (recommended)

```
# point the launcher at your gateway once (local bundled gateway, or a remote URL)
sail-proxy endpoint set local
# or: sail-proxy endpoint set https://gateway.example --key-env SAILPROXY_KEY

# launch Gemini CLI against it
sail-proxy gemini
sail-proxy gemini -m anthropic--claude-4.5-haiku        # any model in the Model Library
sail-proxy gemini --skip-trust -p "summarize this repo"  # headless, one prompt
```

`sail-proxy gemini` sets Gemini CLI's base URL and API key for the session, selects the API-key sign-in type in `~/.gemini/settings.json` (the previous file is backed up beside it — if you also use Gemini CLI with your Google account, switch back with its `/auth` command), and starts the CLI with `gemini-3.5-flash` unless you pass `-m`. Everything after `gemini` goes to Gemini CLI unchanged. Add `--dry-run` to see what would run without changing anything.

Headless runs (`-p`) require a trusted folder: pass `--skip-trust`, or trust the directory once in an interactive session.

Manual configuration

Without the launcher, point Gemini CLI at the gateway with two environment variables and select the API-key sign-in once:

```
export GOOGLE_GEMINI_BASE_URL="https://<your-gateway>/google"
export GEMINI_API_KEY="<your-sail-proxy-api-key>"

gemini            # choose "Gemini API key" when the sign-in dialog appears
```

`GEMINI_API_KEY` alone is not enough: Gemini CLI also needs the API-key sign-in type selected in `~/.gemini/settings.json` (`security.auth.selectedType: "gemini-api-key"`), which the interactive dialog writes for you. `GOOGLE_GEMINI_BASE_URL` should be the gateway's `/google` path with no version segment — Gemini CLI appends `/v1beta/models/...` itself, and the gateway accepts both `v1beta` and `v1`.

Using the `@google/genai` SDK

For your own scripts or agents, configure the SDK's `httpOptions.baseUrl`:

```
import { GoogleGenAI } from '@google/genai';

const ai = new GoogleGenAI({
  apiKey: '<your-sail-proxy-api-key>',
  httpOptions: { baseUrl: 'https://<your-gateway>/google' },
});

const response = await ai.models.generateContent({
  model: 'gemini-2.5-pro',
  contents: 'Say OK',
});
```

The same base URL and key work for `generateContent`, `streamGenerateContent` (streaming) and `embedContent`.

Which models work

Every model listed in the Model Library is reachable through `/google`:

- A **Gemini model that has a deployment** (marked *Deployed* in the Model Library) is served by that deployment directly.
- **Every other model** — Claude, GPT, Mistral, Perplexity, and a Gemini model with no deployment of its own — is served through the platform, the same routing the gateway's OpenAI and Anthropic endpoints use.

You can pass either the bare model name (e.g. `gemini-2.5-pro`) or its `--deployed` id — the gateway resolves whichever one actually serves the request, so Gemini CLI and the SDK see a name they recognize either way.

Embeddings

Call `embedContent` with one text per request (Gemini's `embedContent` carries a single piece of content; there is no batch form here — see Limits below). Usage is metered by the platform exactly as for a chat call, with one exception: an embedding model that exists **only** as a deployment (for example `gemini-embedding-2`) reports no usage at all for `embedContent`, so the gateway estimates the input token count from the request text instead of billing an exact figure.

Image generation

Some Gemini models generate images as well as text. The Model Library marks them with the **image-generation** capability: `gemini-3.1-flash-image` (recommended), `gemini-3-pro-image` and `gemini-2.5-flash-image`. **`gemini-2.5-flash-image` is deprecated by Google and scheduled for shutdown on 2 October 2026** — use `gemini-3.1-flash-image` for new work.

Image output only works on a model's **own deployment** on SAP AI Core (marked *Deployed* in the Model Library) — it is never available through the platform route. Asking an undeployed model for image output gets HTTP 400 in the usual Gemini error shape (HTTP 404 when the model is not listed in the Model Library at all), with the message:

```
Model <model> has no deployment; image output (responseModalities IMAGE) needs a deployment o
f the
model on SAP AI Core.
```

Ask for an image by setting `responseModalities` and, optionally, `imageConfig` in `generationConfig`:

```
const response = await ai.models.generateContent({
  model: 'gemini-3.1-flash-image',
  contents: 'A watercolor painting of a lighthouse at sunset',
  config: {
    responseModalities: ['IMAGE', 'TEXT'],
    imageConfig: { aspectRatio: '1:1' },
  },
});

for (const part of response.candidates[0].content.parts) {
  if (part.inlineData) {
    // part.inlineData.data is base64-encoded image bytes; part.inlineData.mimeType is e.g. i
        mage/png
  }
}
```

`imageConfig.aspectRatio` accepts `1:1`, `1:4`, `1:8`, `2:3`, `3:2`, `3:4`, `4:1`, `4:3`, `4:5`, `5:4`, `8:1`, `9:16`, `16:9` and `21:9`; `imageConfig.imageSize` accepts `512`, `1K`, `2K` and `4K`. Images come back as `inlineData` parts alongside any text parts. `streamGenerateContent` works the same way — the image arrives in the stream's last event.

Data masking does not apply to images. Pseudonymization rewrites text prompts; it is not applied to image prompts or to uploaded (`inlineData`) images, which reach SAP AI Core as you sent them. Do not put personal data in an image prompt or upload an image containing it.

Pricing: SAP publishes no rate for image models, so generated-image tokens are priced from an image output rate an administrator enters in the Model Library (see Admin Cockpit); Google's own list price is a starting point until SAP publishes one. Without a rate, image tokens are priced like ordinary output tokens.

Hooks and PII masking

Everything configured for your other gateway routes — PII masking / pseudonymization and any other request/response hooks — applies to `/google` the same way: request text (system instructions and message parts), response text and streamed text are all covered.

One documented exception: a pseudonymization hook configured for `embedContent` does **not** mask the text being embedded. This matches the OpenAI embeddings endpoint, which runs no masking either — an embedding is a numeric representation of the text, not a chat turn, and masking it would change the vector without protecting anything a downstream consumer could reconstruct from it.

Usage and cost tracking

Requests through `/google` are tracked, entitled and billed exactly like the gateway's other routes — the same API keys, entitlement catalogs, quotas and cost accounting apply. See Manage Access & Monitor Usage with Admin Cockpit.

Limits

- **No token counting.** There is no `:countTokens` equivalent on this route (unlike `/anthropic/v1/messages/count_tokens`).

- **No batch embeddings.** Call `embedContent` once per text.
- **No hosted Gemini tools.** `googleSearch`, `codeExecution` and similar hosted tool entries are refused — only your own function-calling tools work.
- **No file uploads.** Gemini's Files API (`fileData` references) is not available; send the bytes directly as inline image data instead.
- **Non-image inline data is refused.** Inline data parts (`inlineData`) are accepted for images only — audio, video and other inline mime types are rejected.
- **Image output needs a deployment of the model.** A model without its own SAP AI Core deployment cannot return `responseModalities: ["IMAGE"]`, even when it is otherwise reachable through the platform route — see Image generation above.
- **`candidateCount` above 1 is refused.** The gateway always returns exactly one candidate.
- **Safety settings and a few sampling parameters depend on the model.** `safetySettings`, `topK`, `seed`, `presencePenalty` and `frequencyPenalty` reach the model only when a **Gemini model served by its own deployment** answers: your request goes to Google unchanged, so those settings take effect. For every other model the platform ignores them — the request is translated for the platform's own service, which has no equivalent for them, and they are dropped rather than approximated. They are accepted either way; the difference is whether they do anything.
- **`temperature` and `topP` are trimmed to what the model accepts.** Gemini CLI sends both on every request. The platform refuses the pair for Claude models, and refuses `topP` (and any temperature other than 1) for GPT-5 models, so the gateway drops the refused value instead of failing the request — `temperature` wins for Claude; GPT-5 models answer with their defaults. Gemini, Mistral and GPT-4 models receive both unchanged.

Troubleshooting

The gateway answers in Gemini's own error shape (`{"error":{"code","message","status"}}`), so Gemini CLI and the SDK display these directly:

- **`Unsupported Gemini method in "<model>:<method>". This gateway serves generateContent, streamGenerateContent and embedContent.`** — the URL named a method this gateway doesn't implement (for example `:countTokens` or `:batchEmbedContents`). Use one of the three listed methods.
- **`Model <model> is not available through this gateway`** — the model has neither a deployment nor a platform route. Check the spelling against the Model Library, or ask an administrator to deploy it.
- **`Model <model> has no deployment; image output (responseModalities IMAGE) needs a deployment of the model on SAP AI Core.`** — you asked for an image from a model that has no deployment (HTTP 400, or 404 when the model is not listed at all). Ask an administrator to deploy the model; see Image generation.
- **`Model <model> does not support embedContent through this gateway. It has neither an orchestration embedding scenario nor a Google embedding deployment.`** — you called `embedContent` on a model that isn't an embedding model. Pick a model the Model Library marks with embedding capability.
- **`Model <model> is not in your entitlement catalog "<catalog>"`** — the model exists, but your API key's entitlement catalog doesn't include it. Ask an administrator to add it (see Entitlements & Quotas).

For general troubleshooting, see the Troubleshooting guide or the FAQ.